MEK4100 - Midtveisprosjekt

Amer Salimi

26. oktober 2026

Oppgave 1

I denne oppgaven har jeg valgt systemet til en dobbelpendel. Den består av to pendler festet i rekkefølge: den øverste pendelen henger fra en fast feste, mens den nederste pendelen henger fra enden av den øverste. Systemet kan beskrives med 2 frihetsgrader: vinklene θ_1 og θ_2.

Oppgave 2

Under småvinkeltilnærmingen kan vi linearisere bevegelsesligningene for dobbelpendelen, noe som gjør at vi kan finne en analytisk løsning ved hjelp av normalmodusanalysen. For å gjøre uttrykkene enklere, kan vi for eksempel anta at begge pendlene har like masser m og like lengder l. Da reduseres systemet til to lineære, koblede ligninger for de små vinklene θ_1 og $\theta 2$.

Ved små vinkelutslag kan vi bruke tilnærmingene:

$$\sin\theta \approx \theta$$
$$\cos\theta \approx 1$$

De lineære likningene kan skrives på en form som gjør det mulig å finne normalfrekvensene.(Taylor, Kapittel 11.4) I matriseform blir dette:

$$\mathbf{M}\begin{pmatrix}\ddot{\theta}_1\\ \ddot{\theta}_2\end{pmatrix} = -\frac{g}{l}\mathbf{K}\begin{pmatrix}\theta_1\\ \theta_2\end{pmatrix}.$$

hvor $\mathbf{M}$ og $\mathbf{K}$ er en 2×2 matrise.

Normalmodus analyse leder til et karakteristisk polynom med røttene/egenfrekvensene:

$$\omega_1 = \sqrt{\frac{g}{l}\frac{3+\sqrt{5}}{2}}, \quad \omega_2 = \sqrt{\frac{g}{l}\frac{3-\sqrt{5}}{2}}.$$

Finner forholdet mellom vinklene θ_1 og θ_2

$$\frac{\theta_2}{\theta_1} = \alpha = \frac{1+\sqrt{5}}{2}$$

$$\frac{\theta_2}{\theta_1} = \beta = \frac{1-\sqrt{5}}{2}$$

Den analytiske løsningen for (en liten vinkelutslag) systemet under tilnærmingen er da:

$$\theta_1(t) = C_1 \cos\big(\omega_1 t + \phi_1\big) \; + \; C_2 \cos\big(\omega_2 t + \phi_2\big),$$

$$\theta_2(t) = \alpha\, C_1 \cos\big(\omega_1 t + \phi_1\big) \; + \; \beta\, C_2 \cos\big(\omega_2 t + \phi_2\big),$$

Oppgave 3

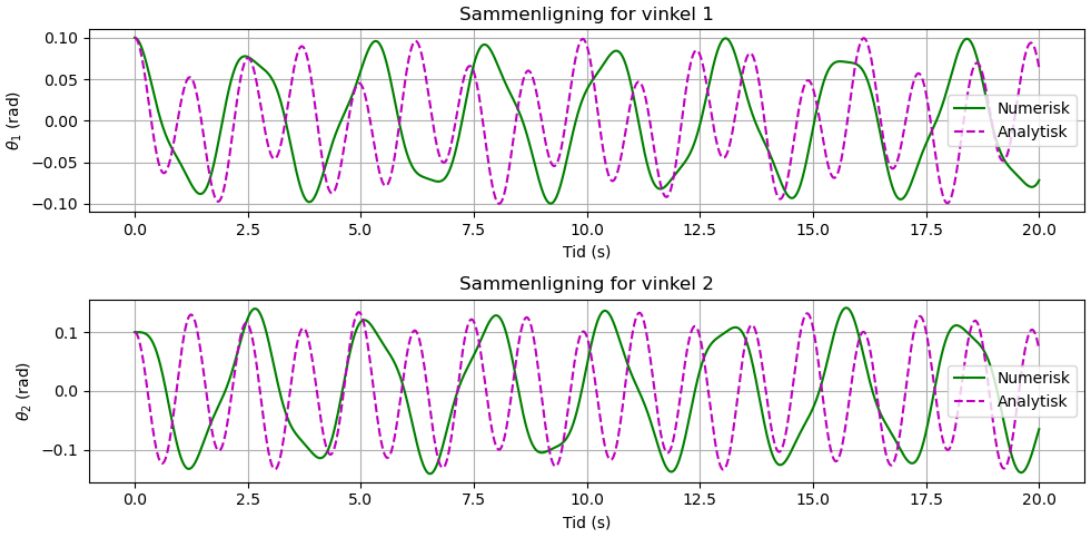
Beregner egenfrekvensene og bruker egenvektorforholdene α og β. Løser koeffisientene C_1 og C_2.

$$\theta_1(0) = 0.1, \quad \theta_2(0) = 0.1, \quad \dot{\theta}_1(0) = \dot{\theta}_2(0) = 0$$

$$C_1 + C_2 = 0.1, \quad \alpha C_1 + \beta C_2 = 0.1.$$

$$C_1 \approx 0.07236, \quad C_2 \approx 0.02764$$

Figuren under viser både den numeriske løsningen (grønn) og den analytiske løsningen (lilla) for de to vinklene som funksjon av tid. De lineære differensiallikningene løses numerisk ved å oppdatere akselerasjonen, farten og til slutt vinklene. Observerer at for småvinkel tilnærminger så er disse to løsningene ganske like. Python-kode for simuleringene er lagt til nederst, med kommentarer.



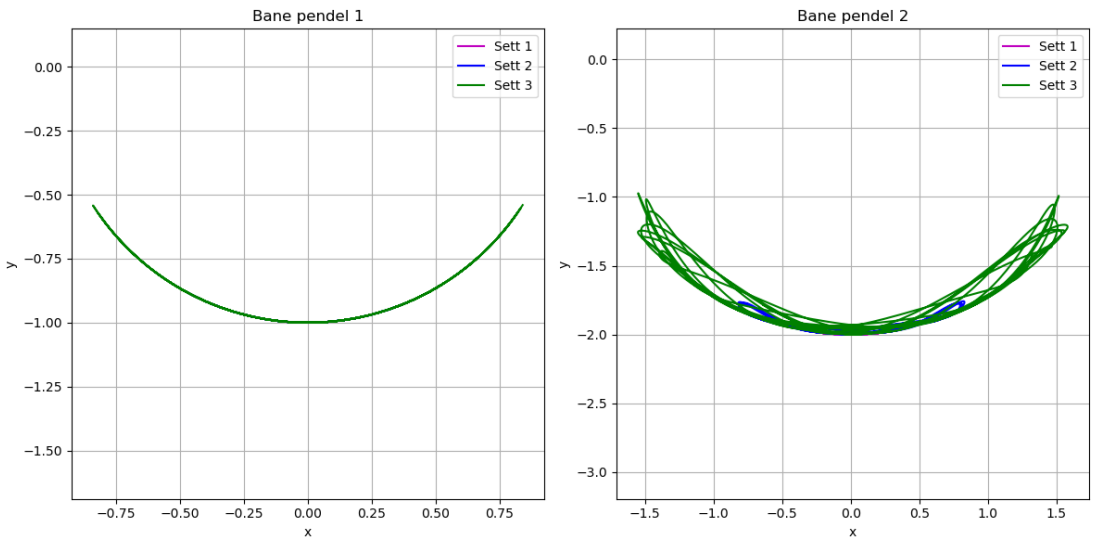
Jeg har brukt 3 sett med initialbetingelser $(\theta_1, \theta_2, \omega_1, \omega_2)$, ved $t = 0$, til neste sammenligning av bevegelsen, hvor Lagrange-likningene blir løst numerisk for det generelle tilfellet uten tilnærminger:

Sett 1: $\theta_1(0) = 0.1 \quad \theta_2(0) = 0.1 \quad \omega_1(0) = 0.0 \quad \omega_2(0) = 0.0$

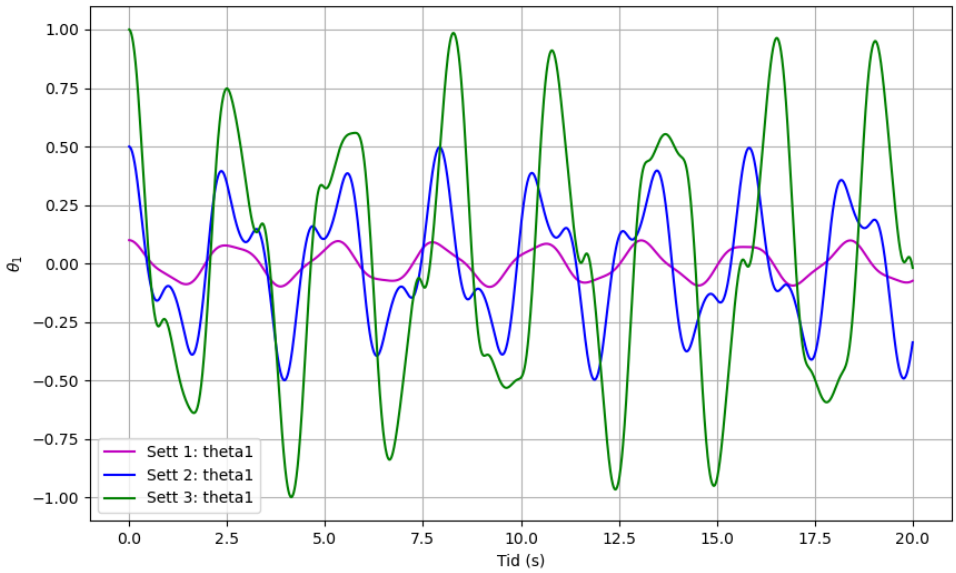
Sett 2: $\theta_1(0) = 0.5 \quad \theta_2(0) = 0.2 \quad \omega_1(0) = 0.0 \quad \omega_2(0) = 0.0$

Sett 3: $\theta_1(0) = \pi \quad \theta_2(0) = 0.3 \quad \omega_1(0) = 0.0 \quad \omega_2(0) = 0.0$

Figur 1 viser banene til pendlene i xy-planet for å vise hvordan bevegelsen varierer med initialbetingelsene. Figur 2 viser $\theta_1(t)$ for de tre tilfellene, for å se utviklingen av bevegelsen over tid.



Figur 1: Bane i xy-planet



Figur 2: $\theta_1(t)$ for forskjellig inititalbetingelser

Observerer og ser at for små utslag(lilla) er pendelen nær harmonisk og den analytiske løsningen er en god tilnærming. Når initialvinklene økes (blå) blir effekten av de ikke-linære leddene merkbare, bevegelsen er fortsatt periodisk men merker avvik fra den analytiske løsningen. For store utslag(grønn) dominerer ikke-lineære leddene betydelig, med stor avvik fra den analytiske løsningen. Ser også at banene i xy-planet (Figur 1) blir komplekse med ikke periodiske svingninger.

```python
#Oppgave 3a)

import numpy as np
import matplotlib.pyplot as plt

# parametere
g = 9.81      # tyngdeakselerasjon
m = 1.0       # masse     # m1=m2
l = 1.0       # lengde    l1=l2

# Initialbetingelser (sm  vinkler)
omega1_0 = 0.0
omega2_0 = 0.0

theta1_0 = 0.1   # i radianer(dimensjonsl st)
theta2_0 = 0.1

# Tidsskritt og total tid
dt = 0.01
T = 20
N = int(T/dt)
time_arr = np.linspace(0, T, N)

# Arrays
theta1_num = np.zeros(N)
theta2_num = np.zeros(N)

# Initialiserer variablene
theta1 = theta1_0
theta2 = theta2_0
omega1 = omega1_0
omega2 = omega2_0

# Euler-Cromer
for i in range(N):
    theta1_num[i] = theta1
    theta2_num[i] = theta2

    a1 = - (g/l) * (2*theta1 - theta2)      # akselerasjoner
    a2 =   (g/l) * (2*theta1 - 2*theta2)

    # Oppdaterer vinkelhastigheter
    omega1 += a1 * dt
    omega2 += a2 * dt

    # Oppdaterer vinkler med de nye vinkelhastighetene
    theta1 += omega1 * dt
    theta2 += omega2 * dt



# Anlytisk l sning (sm vinkeltiln rming)

```

```
54 # Egenfrekvenser
55 omega1A = np.sqrt((g/l) * (3 + np.sqrt(5)) / 2)
56 omega2A = np.sqrt((g/l) * (3 - np.sqrt(5)) / 2)
57
58 # Forhold til egenvektorene
59 alpha = (1 + np.sqrt(5)) / 2
60 beta  = (1 - np.sqrt(5)) / 2
61
62 C1 = (theta1_0 * (1 - beta)) / (alpha - beta)
63 C2 = theta1_0 - C1
64
65 # Anaytiske l sningene
66 theta1_A_arr = C1 * np.cos(omega1A * time_arr) + C2 * np.cos(omega2A *
       time_arr)
67 theta2_A_arr = alpha * C1 * np.cos(omega1A * time_arr) + beta * C2 * np.
       cos(omega2A * time_arr)
68
69
70
71 plt.figure(figsize=(10, 5))
72
73 # Plott for theta 1
74 plt.subplot(2,1,1)
75 plt.plot(time_arr, theta1_num, 'g-', label='Numerisk')
76 plt.plot(time_arr, theta1_A_arr, 'm--', label='Analytisk')
77 plt.xlabel("Tid (s)")
78 plt.ylabel(r"$\theta_1$ (rad)")
79 plt.title("Sammenligning for vinkel 1")
80 plt.legend()
81 plt.grid(True)
82
83 # Plott for theta 2
84 plt.subplot(2,1,2)
85 plt.plot(time_arr, theta2_num, 'g-', label='Numerisk')
86 plt.plot(time_arr, theta2_A_arr, 'm--', label='Analytisk')
87 plt.xlabel("Tid (s)")
88 plt.ylabel(r"$\theta_2$ (rad)")
89 plt.title("Sammenligning for vinkel 2")
90 plt.legend()
91 plt.grid(True)
92
93 plt.tight_layout()
94 plt.show()
```

```python
#Oppgave 3b
import numpy as np
import matplotlib.pyplot as plt

g  = 9.81              #tyngdeaskelerasjon
m1 = 1.0               #lik masse og lengde
m2 = 1.0
l1 = 1.0
l2 = 1.0



def M(q):                                                  # Returnerer massetermen M(q) for dobbelpendelen. q = [theta1, theta2]
    theta1, theta2 = q
    M11 = (m1 + m2) * l1**2
    M12 = m2 * l1 * l2 * np.cos(theta1 - theta2)
    M21 = M12
    M22 = m2 * l2**2
    return np.array([[M11, M12],
                     [M21, M22]])

def b(q, v):
    theta1, theta2 = q
    omega1, omega2 = v

    C_v_1 = - m2 * l1 * l2 * np.sin(theta1 - theta2) * (omega2**2)
      #Coriolis, sentrifugalledd = m2*l1*l2*sin(theta1-theta2) * omega^2-termene
    C_v_2 =   m2 * l1 * l2 * np.sin(theta1 - theta2) * (omega1**2)

    g_1 = (m1 + m2) * g * l1 * np.sin(theta1)
       #tyngdekrefter = (m1+m2)*g*l1*sin(theta1) og m2*g*l2*sin(theta2)
    g_2 = m2 * g * l2 * np.sin(theta2)

    b1 = C_v_1 - g_1
    b2 = C_v_2 - g_2
    return np.array([b1, b2])

def euler_cromer_step(q, v, dt):
    a = np.linalg.inv(M(q)) @ b(q, v)                          #regner ut a=M(q)^(-1) * b(q,v) og oppdaterer hastigheter og vinkler

    v_new = v + a * dt

    q_new = q + v_new * dt
    return q_new, v_new

def S_dobbelpendel(q0, v0, t0, t_slutt, dt):          #Her simuleres dobbelpendelen med de gitte initialbetingelsene. For hver initialbetingelse simuleres bevegelsen over en gitt tidsperiode, og resultatene lagres.

```

```
    nsteps = int((t_slutt - t0) / dt)
    q_H = np.zeros((nsteps, 2))
    v_hist = np.zeros((nsteps, 2))
    t_hist = np.linspace(t0, t_slutt, nsteps)

    q = q0.copy()
    v = v0.copy()

    for i in range(nsteps):
        q_H[i] = q
        v_hist[i] = v
        q, v = euler_cromer_step(q, v, dt)

    return t_hist, q_H, v_hist

def posisjoner(q_H):                                    # posisjon (x, y) for
    hver pendel

    theta1 = q_H[:, 0]
    theta2 = q_H[:, 1]

    x1 = l1 * np.sin(theta1)
    y1 = -l1 * np.cos(theta1)

    x2 = x1 + l2 * np.sin(theta2)
    y2 = y1 - l2 * np.cos(theta2)
    return x1, y1, x2, y2




t0 = 0.0
t_slutt = 20.0
dt = 0.01

# Sett 1:
q0_1 = np.array([0.1, 0.1])
v0_1 = np.array([0.0, 0.0])

# Sett 2:
q0_2 = np.array([0.5, 0.2])
v0_2 = np.array([0.0, 0.0])

# Sett 3:
q0_3 = np.array([1.0, 0.5])
v0_3 = np.array([0.0, 0.0])


t1, q_H1, _ = S_dobbelpendel(q0_1, v0_1, t0, t_slutt, dt)
t2, q_H2, _ = S_dobbelpendel(q0_2, v0_2, t0, t_slutt, dt)
t3, q_H3, _ = S_dobbelpendel(q0_3, v0_3, t0, t_slutt, dt)


```

```
x1_1, y1_1, x2_1, y2_1 = posisjoner(q_H1)                    # Beregner posisjoner for hvert tilfelle
x1_2, y1_2, x2_2, y2_2 = posisjoner(q_H2)
x1_3, y1_3, x2_3, y2_3 = posisjoner(q_H3)



plt.figure(figsize=(12, 6))

# Banene til pendel 1
plt.subplot(1, 2, 1)
plt.plot(x1_1, y1_1, 'm-', label='Sett 1')
plt.plot(x1_2, y1_2, 'b-', label='Sett 2')
plt.plot(x1_3, y1_3, 'g-', label='Sett 3')
plt.xlabel("x")
plt.ylabel("y")
plt.title("Bane pendel 1")
plt.legend()
plt.grid(True)
plt.axis("equal")

# Banene til pendel 2
plt.subplot(1, 2, 2)
plt.plot(x2_1, y2_1, 'm-', label='Sett 1')
plt.plot(x2_2, y2_2, 'b-', label='Sett 2')
plt.plot(x2_3, y2_3, 'g-', label='Sett 3')
plt.xlabel("x")
plt.ylabel("y")
plt.title("Bane pendel 2")
plt.legend()
plt.grid(True)
plt.axis("equal")

plt.tight_layout()
plt.show()


plt.figure(figsize=(10, 6))
plt.plot(t1, q_H1[:,0], 'm-', label='Sett 1: theta1')
plt.plot(t2, q_H2[:,0], 'b-', label='Sett 2: theta1')
plt.plot(t3, q_H3[:,0], 'g-', label='Sett 3: theta1')
plt.xlabel("Tid (s)")
plt.ylabel(r"$\theta_1$")
plt.legend()
plt.grid(True)
plt.show()


"""
brukt chatGPT til noe hjelp med syntaks og feils king.
"""
```